Question 2: Provision scalable infrastructure using Terraform (Infrastructure as Code): write configuration files, initialize, plan, and apply the infrastructure.

Course outcome: CO2, CO3

Terraform with the docker provider provisioned nginx web containers as code. The full workflow was demonstrated: init, plan, apply (2 replicas), then scaling to 4 by changing one variable and re-applying.

Configuration (terraform/main.tf)

```
# Lab 3 - provision scalable infrastructure with Terraform (docker provider)
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

provider "docker" {}

variable "replicas" {
  description = "Number of web container replicas"
  type        = number
  default     = 2
}

resource "docker_image" "web" {
  name = "nginx:alpine"
}

resource "docker_container" "web" {
  count = var.replicas
  name  = "lab3-web-${count.index + 1}"
  image = docker_image.web.image_id

  ports {
    internal = 80
    external = 8082 + count.index
  }
}

output "container_names" {
  value = [for c in docker_container.web : c.name]
}

output "container_urls" {
  value = [for c in docker_container.web : "http://localhost:${c.ports[0].external}"]
}
```

Workflow

```
$ terraform init      # downloads the kreuzwerker/docker provider
Terraform has been successfully initialized!

$ terraform plan -out=tfplan
Plan: 2 to add, 0 to change, 0 to destroy.

$ terraform apply tfplan
docker_container.web[0]: Creating...
docker_container.web[1]: Creating...
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

$ docker ps --filter name=lab3-web
lab3-web-2   0.0.0.0:8083->80/tcp
lab3-web-1   0.0.0.0:8082->80/tcp
```

Scaling 2 -> 4

```
$ terraform apply -var="replicas=4" -auto-approve
docker_container.web[2]: Creating...
docker_container.web[3]: Creating...
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

$ terraform state list
docker_container.web[0..3]
docker_image.web
```

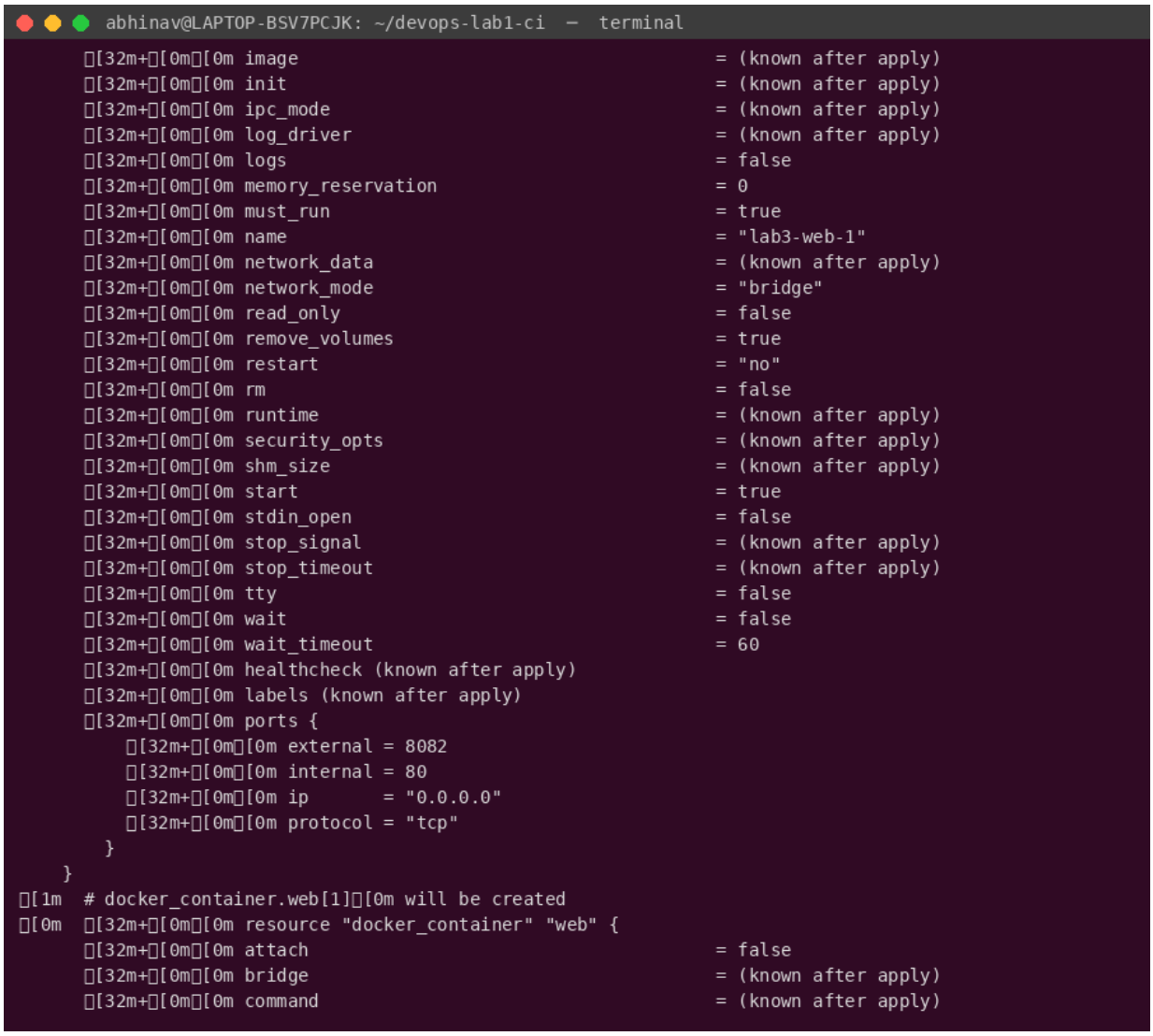


Fig: plan/apply with 2 containers

Observations

- Terraform's plan output showed exactly what would be created before applying - no surprises.
- Scaling was a one-variable change; Terraform computed the delta (2 to add, 0 to change) itself.
- Each container got the next free host port (8082..8085) from count.index arithmetic.
- The hashicorp apt key failed verification, so the terraform binary was installed from releases.hashicorp.com directly.

Conclusion

All experiments were performed on Ubuntu 24.04 LTS with Ansible, Terraform, Prometheus, Grafana, the ELK stack and Selenium. The evidence (command logs, configuration files, screenshots and commit history) is committed in the GitHub repository github.com/theabhinav0231/devops-lab-3 under the docs/ directory.